

CAPA TEMELLERİ

[Kaynak](#)

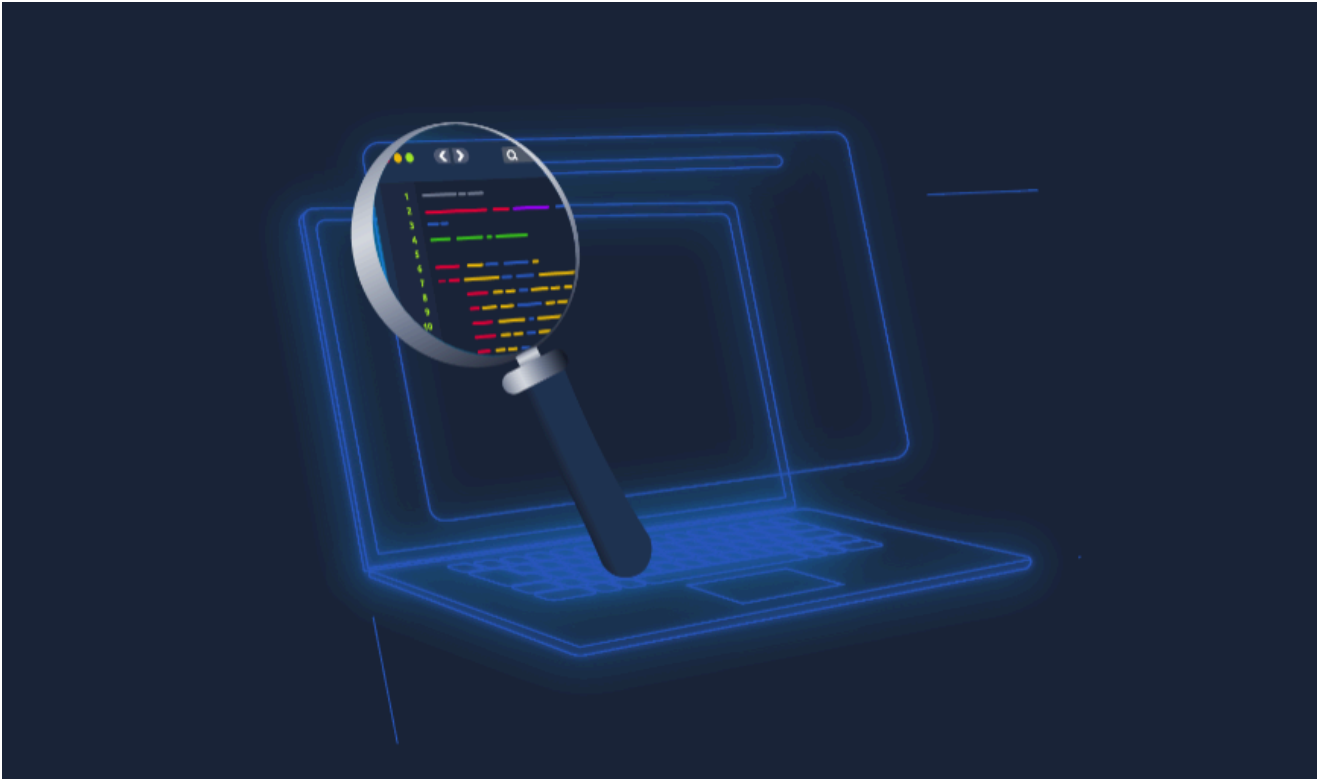
İÇERİK

1. CAPA ile Statik Kötü Amaçlı Yazılım Analizi (Static Malware Analysis)
2. CAPA Kullanımı ve Analiz Çıktılarının Yorumlanması
3. CAPA Analiz Çıktılarının Derinlemesine İncelenmesi - Bölüm 1: Metadata, MITRE ve MAEC
4. CAPA Sonuçlarının İncelenmesi Bölüm 2: Malware Behavior Catalogue (MBC)
5. CAPA Sonuçlarının İncelenmesi Bölüm 3: Namespaces (İsim Uzayları)
6. CAPA Sonuçlarının İncelenmesi Bölüm 4: Capability (Yetenekler) ve YAML Mimarisi
7. CAPA: Zararlı Yazılım Analizinde İleri Seviye Kural Tespitleri ve Very Verbose (Çok Detaylı) Çıktı Yönetimi

CAPA ile Statik Kötü Amaçlı Yazılım Analizi (Static Malware Analysis)

Kötü Amaçlı Yazılım Analizinin Temelleri ve Risk Faktörleri

Potansiyel olarak zararlı olabilecek bir yazılımı (malware) analiz ederken karşılaşılan en büyük operasyonel risk, analizin yapıldığı makinenin veya ağ ortamının enfekte olmasıdır. Bu riski izole etmek için yazılımlar genellikle bir **sandbox** (sistemin geri kalanından tamamen yalıtılmış, sanallaştırılmış ve davranışların loglandığı güvenli test ortamı) içerisinde çalıştırılır.



Güvenlik literatüründe malware analizi temel olarak iki ana disipline ayrılır:

1. **Dynamic Analysis** (Dinamik Analiz): Zararlı yazılımın izole bir ortamda bilfiil çalıştırılarak sistem üzerindeki anlık davranışlarının (dosya oluşturma, ağ bağlantıları, kayıt defteri değişiklikleri) izlenmesi sürecidir.
2. **Static Analysis** (Statik Analiz): Zararlı yazılımın **çalıştırılmadan**, salt dosya yapısı, kaynak kodu, string ifadeleri veya makine kodunun incelenmesi işlemidir. Bu yöntem, dosyanın kapasitesini ve iç yapısını anlamak için kritik bir ön aşamadır.

Bu çalışma notu, **FireEye Mandiant** ekibi tarafından geliştirilen **CAPA** aracı kullanılarak gerçekleştirilen gelişmiş **Static Analysis** metodolojisine odaklanmaktadır.

CAPA (Common Analysis Platform for Artifacts) Nedir ve Nasıl Çalışır?

CAPA, bir dosyanın veya çalıştırılabilir (executable) bir programın hangi yeteneklere sahip olduğunu tespit etmek için tasarlanmış, kural tabanlı (rule-based) ve otomatikleştirilmiş bir analiz motorudur. Temel amacı, analistin manuel olarak saatler harcayacağı bir süreci saniyeler içine sığdırarak dosyanın genel profilini ve niyetini ortaya çıkarmaktır.

Desteklenen Analiz Hedefleri (Artifacts)

CAPA, yalnızca belirli bir formata bağlı kalmaz; modern sistemlerde karşılaşılan birçok farklı çalıştırılabilir yapıyı analiz edebilir:

- **PE (Portable Executables)**: Windows işletim sistemlerinde kullanılan standart çalıştırılabilir dosya formatlarıdır (Örn: .exe , .dll , .sys).
- **ELF binaries (Executable and Linkable Format)**: Linux ve Unix tabanlı sistemlerin standart çalıştırılabilir dosya ve kütüphane formatıdır.
- **.NET Modules**: Microsoft'un .NET framework'ü kullanılarak derlenmiş uygulamalar.
- **Shellcode**: Genellikle bir zafiyet sömürüldükten (exploitation) sonra belleğe doğrudan enjekte edilen ve sistemde bir komut satırı (shell) açmayı hedefleyen spesifik makine kodu parçacıkları.
- **Sandbox Reports**: Dinamik analiz araçlarından elde edilen davranışsal çıktı raporları.

Arka Plan Çalışma Mantığı ve Tespit Kapasitesi

CAPA, analiz edilen dosyayı körü körüne incelemez. Arka planda, malware yazarlarının sıklıkla kullandığı davranış biçimlerini tanımlayan geniş bir kural seti (ruleset) ile dosyayı eşleştirir. Bir programın kaynak kodunda veya derlenmiş yapısında aşağıdaki gibi kritik yetenekler aranır:

- **Network Communication** (Ağ İletişimi): Dosya dışarıya bir C2 (Command and Control) sunucusuna bağlanma, HTTP istekleri yapma veya belirli portları dinleme yeteneğine sahip mi?

- **File Manipulation** (Dosya Manipülasyonu): Dosya okuma, yazma, silme veya kendini başka bir dizine kopyalama gibi sisteme müdahale eden eylemler var mı?
- **Process Injection** (Süreç Enjeksiyonu): Zararlı yazılımın kendi kodunu veya payload'unu, işletim sistemindeki meşru bir sürecin (örn: `explorer.exe`) bellek alanına enjekte ederek güvenlik araçlarından gizlenmeye (Evasion) çalışma yeteneği.

ÖNEMLİ: CAPA'nın sektörel anlamda bu kadar kritik olmasının sebebi, yıllar süren **Reverse Engineering** (Tersine Mühendislik: Derlenmiş ve makine diline çevrilmiş bir yazılımın, nasıl çalıştığını anlamak için decompile/disassemble edilerek analiz edilmesi süreci) bilgi birikimini otomatik bir araca entegre etmiş olmasıdır.

Sektörel Kullanım Alanları ve Operasyonel Değeri

Tersine mühendislik alanında uzman olmayan bir siber güvenlik analisti bile CAPA kullanarak karmaşık bir zararlının ne yapabileceğini hızlıca kavrayabilir. Bu durum özellikle aşağıdaki operasyonlarda hayati rol oynar:

- **Malware Analysis:** Zararlının kapasitesini hızlıca profillemek.
- **Threat Hunting** (Tehdit Avcılığı): Ağ veya sistemler üzerinde proaktif olarak zararlı aktiviteleri ararken, şüpheli binary (ikili) dosyaların yeteneklerini hızlıca sınıflandırmak.
- **Incident Response** (Olay Müdahalesi): Bir sistem ihlali (breach) anında, saldırganın bıraktığı zararlının nelere kadir olduğunu (veri mi çalıyor, sistemi mi şifreliyor, backdoor mu açıyor?) anında tespit edip defansif önlemleri buna göre kurulumak.

CAPA Kullanımı ve Analiz Çıktılarının Yorumlanması

CAPA'nın Çalıştırılması ve Temel İşleyiş Mantığı

CAPA'nın operasyonel kullanımı oldukça basittir ancak arka planda yürütülen kural eşleştirme (rule-matching) süreci ciddi bir işlem gücü ve zaman gerektirir. Aracı çalıştırmak için öncelikle hedef dosyanın (binary) ve CAPA yürütülebilir dosyasının bulunduğu dizine geçilmelidir.

Aşağıdaki komutta, zararlı olduğundan şüphelenilen **cryptbot.bin** isimli **PE (Portable Executable)** dosyası statik analize tabi tutulmaktadır.

```
PS C:\Users\Administrator\Desktop\capa> capa.exe .\cryptbot.bin
loading : 100%|██████████████████████| 485/485 [00:00<00:00, 1108.84
rules/s]/ analyzing program...
```

Bu komutu çalıştırdığımızda ekrana gelen "loading" ve "analyzing program" ibareleri kritik bir arka plan sürecini ifade eder. CAPA bu aşamada, dosyanın makine kodunu, string (metin) ifadelerini, içe/dışa aktarılan fonksiyonlarını (imports/exports) ve PE başlıklarını (headers) okur. Ardından, belleğe yüklediği yüzlerce farklı davranışsal kuralı (örneğin 485 kural) bu verilerle eşleştirmeye çalışır. Dosyanın boyutuna ve karmaşıklığına bağlı olarak bu analiz

dakikalar sürebilir. Eğer bu adımda bir bellek hatası veya uyumsuzluk alırsak (B Planı), dosyanın "packed" (paketlenmiş/sıkıştırılmış) olup olmadığını kontrol etmemiz ve önce "unpacking" (paketi açma) işlemi uygulamamız gerekir.

CAPA Parametreleri ve Analiz Derinliğinin Ayarlanması

Varsayılan `capa.exe <dosya>` komutu sadece özet tabloları döndürür. Ancak tersine mühendislik süreçlerinde bir kuralın *neden* tetiklendiğini, zararlıının tam olarak hangi hafıza adresinde (offset) veya fonksiyon çağrısında bu davranışı sergilediğini bilmek hayati önem taşır. Bu derinliği sağlamak için aşağıdaki flag'ler (parametreler) kullanılır:

Parametre	Uzun Versiyon	Teknik Açıklama ve Kullanım Mantığı	Örnek Sözdizimi (Syntax)
-h	--help	Aracın kabul ettiği tüm parametreleri ve argümanları listeler. Operasyonel ortamda aracın desteklediği çıktı formatlarını (örn: JSON çıktısı almak için) öğrenmek için başvurulur.	<code>capa -h</code>
-v	--verbose	Verbose Mode (Ayrıntılı Mod): Yalnızca yetenekleri listelemekle kalmaz, bu yeteneklerin dosyanın hangi bölümünde (fonksiyon, string, vb.) tespit edildiğini mantıksal açıklamalarıyla birlikte sunar.	<code>capa.exe .\cryptbot.bin -v</code>
-vv	--vverbose	Very Verbose Mode (Çok Ayrıntılı Mod): Bir kuralın eşleşmesi için gereken tam makine kodu (assembly) talimatlarını, offset adreslerini ve eşleşme ağaçlarını (matching trees) döküm olarak verir. Doğrudan IDA Pro veya Ghidra gibi decompiler araçlarıyla entegre çalışacak analistler için şarttır.	<code>capa.exe .\cryptbot.bin -vv</code>

CAPA Varsayılan Analiz Çıktısının Satır Satır İncelenmesi

Analiz tamamlandığında CAPA, terminale hiyerarşik tablolar halinde bir rapor sunar. Bu rapor, zararlıının anatomisini ortaya çıkaran nihai haritadır.

```
PS C:\Users\Administrator\Desktop\capa> capa .\cryptbot.bin
```

md5	3b9d26d2e7433749f2c32edb13a2b0a2
sha1	969437df8f4ad08542ce8fc9831fc49a7765b7c5
sha256	ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c
analysis	static
os	windows
format	pe
arch	i386
path	/home/strategos/Room-CAPA/cryptbot.bin

Temel Dosya Bilgileri (Metadata) Yorumu:

Çıktının ilk tablosu dosyanın kimlik kartıdır.

- **md5, sha1, sha256:** Dosyanın kriptografik özetleridir (hash). **ÖNEMLİ:** Bu hash değerleri, zararlının daha önce tespit edilip edilmediğini doğrulamak için doğrudan **VirusTotal** veya **AlienVault OTX** gibi Threat Intelligence (Tehdit İstihbaratı) platformlarında sorgulanmalıdır.
- **os: windows / format: pe / arch: i386:** Bu satırlar bize dosyanın 32-bit (i386) bir Windows Portable Executable (.exe, .dll vb.) olduğunu kesin olarak kanıtlar. Bu bilgi sayesinde dinamik analiz için kuracağımız sanal makinenin mimarisini (32-bit Windows) netleştirmiş oluyoruz.

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Obfuscated Files or Information [T1027] Obfuscated Files or Information::Indicator Removal from Tools [T1027.005] Virtualization/Sandbox Evasion::System Checks [T1497.001]
DISCOVERY	File and Directory Discovery [T1083]
EXECUTION	Command and Scripting Interpreter::PowerShell [T1059.001] Shared Modules [T1129]
IMPACT	Resource Hijacking [T1496]
PERSISTENCE	Scheduled Task/Job::At [T1053.002] Scheduled Task/Job::Scheduled Task [T1053.005]

MITRE ATT&CK Çerçevesi Yorumu:

Bu tablo, teknik bulguları Blue Team (Savunma Ekibi) için anlamlı eylem planlarına dönüştürür.

- **DEFENSE EVASION (Savunmadan Kaçınma):** [T1497.001] tekniği zararlının bir Sandbox veya Sanal Makine (VM) içinde olup olmadığını kontrol ettiğini gösteriyor. Eğer VM tespit ederse çalışmayı durduracaktır.
- **EXECUTION & PERSISTENCE (Çalıştırma ve Kalıcılık):** Zararlının sistemde **PowerShell** komutları yürütebildiğini [T1059.001] ve kurban bilgisayarını her yeniden başlatıldığında otomatik çalışmak için Zamanlanmış Görevler (Scheduled Tasks - [T1053]) oluşturduğunu anlıyoruz. Olay müdahale (Incident Response) sırasında ilk bakacağımız yer Windows Task Scheduler olmalıdır.

- **IMPACT (Etki):** Resource Hijacking [T1496] genellikle zararlının kurbanın CPU/GPU kaynaklarını kullanarak kripto para madenciliği (cryptomining) yaptığının en büyük göstergesidir. Dosyanın adının "cryptbot" olması bu bulguyu doğrular niteliktedir.

MAEC Category	MAEC Value
malware-category	launcher

MBC Objective	MBC Behavior
ANTI-BEHAVIORAL ANALYSIS	Virtual Machine Detection [B0009]
ANTI-STATIC ANALYSIS	Executable Code Obfuscation::Argument Obfuscation [B0032.020] Executable Code Obfuscation::Stack Strings [B0032.017]
COMMUNICATION	HTTP Communication [C0002] HTTP Communication::Read Header [C0002.014]
DATA	Check String [C0019] Encode Data::Base64 [C0026.001] Encode Data::XOR [C0026.002]
DEFENSE EVASION	Obfuscated Files or Information::Encoding-Standard Algorithm [E1027.m02]
DISCOVERY	File and Directory Discovery [E1083]
EXECUTION	Command and Scripting Interpreter [E1059]
FILE SYSTEM	Create Directory [C0046] Delete File [C0047] Read File [C0051] Writes File [C0052]
MEMORY	Allocate Memory [C0007]
PROCESS	Create Process [C0017]

Malware Behavior Catalog (MBC) Yorumu:

- **ANTI-STATIC ANALYSIS:** Stack Strings kullanımı, zararlı yazılım yazarının string ifadelerini (URL'ler, IP'ler, dosya yolları) açık metin olarak bırakmak yerine analizciden gizlemek için her bir karakteri bellekte (stack) çalışma zamanında (runtime) birleştirdiğini gösterir.
- **DATA (Veri Manipülasyonu):** C2 sunucusuyla iletişim kurarken veya konfigürasyon dosyalarını saklarken verileri gizlemek için **Base64** ve **XOR** şifreleme algoritmalarını kullandığını net bir şekilde görüyoruz.
- **FILE SYSTEM & PROCESS:** Zararlı kendi kopyasını veya indireceği diğer modülleri (payload) kaydetmek için klasör oluşturabiliyor, dosya yazıp silebiliyor ve yeni süreçler (process) başlatabiliyor.

Capability	Namespace
reference anti-VM strings	anti-analysis/anti-vm/vm-detection
reference anti-VM strings targeting VMWare	anti-analysis/anti-vm/vm-detection
reference anti-VM strings targeting VirtualBox	anti-analysis/anti-vm/vm-detection
contain obfuscated stackstrings (2 matches)	anti-analysis/obfuscation/string/stackstring
reference HTTP User-Agent string	communication/http
check HTTP status code	communication/http/client
reference Base64 string	data-manipulation/encoding/base64
encode data using XOR	data-manipulation/encoding/xor
contain a thread local storage (.tls) section	executable/pe/section/tls
get common file path	host-interaction/file-system
create directory	host-interaction/file-system/create
delete file	host-interaction/file-system/delete
read file on Windows (4 matches)	host-interaction/file-system/read
write file on Windows (5 matches)	host-interaction/file-system/write
get thread local storage value	host-interaction/process
create process on Windows	host-interaction/process/create
allocate or change RWX memory	host-interaction/process/inject
reference cryptocurrency strings	impact/cryptocurrency
link function at runtime on Windows (5 matches)	linking/runtime-linking
parse PE header (4 matches)	load-code/pe
resolve function by parsing PE exports (186 matches)	load-code/pe
run PowerShell expression	load-code/powershell/
schedule task via at	persistence/scheduled-tasks
schedule task via schtasks	persistence/scheduled-tasks

Spesifik Yetenekler (Capabilities) Yorumu:

Bu bölüm CAPA'nın en değerli çıktısıdır; çünkü kütüphane çağrıları (API calls) üzerinden makinenin tam olarak ne yapmaya programlandığını söyler.

- **VMWare ve VirtualBox tespiti:** Zararlı, MAC adresleri, sürücü dosyaları veya kayıt defteri anahtarları üzerinden bu iki sanallaştırma platformunu hedef alan spesifik kontroller yapıyor. Dinamik analiz yaparken bu VM'leri değil, bare-metal (fiziksel) bir cihazı veya çok iyi izole edilmiş/maskelenmiş özel bir hypervisor kullanmamız gerektiği anlamına gelir.
- **allocate or change RWX memory: DİKKAT!** RWX (Read, Write, Execute - Okuma, Yazma, Çalıştırma) yetkisine sahip bir bellek alanı tahsis edilmesi, %99 oranında **Process Injection** (Süreç Enjeksiyonu) veya **Unpacking** (Kendi kendini bellekte açma) aktivitesine işaret eder. Zararlı, zararsız görünen başka bir sürecin belleğine sızarak oradan çalışmaya niyetlidir.
- **resolve function by parsing PE exports (186 matches):** Dinamik API çözümlemesi yapıyor. Yani kullanacağı Windows API kütüphanelerini IAT (Import Address Table) tablosunda açıkça belirtmek yerine, çalışma zamanında zararlının kendisi bularak yüklüyor (Runtime Linking). Bu, statik analiz araçlarını (ve antivirüsleri) atlatmak için kullanılan klasik bir tekniktir.

Alternatif Analiz Yöntemi: Önceden İşlenmiş Logların Okunması

Büyük boyutlu binary dosyalarının CAPA analizleri makineyi kilitleyebilir veya çok uzun sürebilir. Operasyonel verimlilik adına, daha önce analiz edilmiş ve bir metin dosyasına yönlendirilmiş (örneğin `capa.exe target.bin > output.txt` komutuyla) sonuçları sisteme yüklemek hız kazandırır.

```
PS C:\Users\Administrator\Desktop\capa> Get-Content .\cryptbot.txt
```

Bu Powershell komutu, Linux'taki `cat` komutunun karşılığıdır. `Get-Content`, önceden oluşturulmuş `cryptbot.txt` dosyasının içeriğini belleğe alarak terminal ekranına basar. Böylece saatler sürebilecek bir kural analizini beklemek yerine saniyeler içinde yukarıda yorumladığımız aynı hiyerarşik rapora ulaşp doğrudan istihbarat çıkarma (IOC extraction) ve analiz safhasına geçebiliriz.

CAPA Analiz Çıktılarının Derinlemesine İncelenmesi - Bölüm 1: Metadata, MITRE ve MAEC

Bu çalışma notunda, **CAPA** aracının şüpheli bir binary (ikili dosya) üzerinde yürüttüğü statik analiz sonucunda ürettiği hiyerarşik çıktının ilk aşamasını teknik bir mikroskop altına alıyoruz. Bir siber güvenlik analisti olarak bu blokları sadece okumakla kalmamalı, arka plandaki işletim sistemi mimarisi ve saldırgan (Threat Actor) motivasyonları ile bağdaştırmalıyız.

Temel Dosya Bilgileri (Metadata) ve Analiz Bağlamı

Analizin ilk bloğu, incelenen dosyanın kimlik kartı niteliğindedir. Bu bölüm, zararlıının mimarisini, analiz yöntemimizi ve dosyanın kriptografik kimliğini ortaya koyar.

md5	3b9d26d2e7433749f2c32edb13a2b0a2
sha1	969437df8f4ad08542ce8fc9831fc49a7765b7c5
sha256	ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c
analysis	static
os	windows
format	pe
arch	i386
path	/home/strategos/Room-CAPA/cryptbot.bin

Bu terminal çıktısının arka plandaki teknik anlamı ve operasyonel değeri şu şekildedir:

Zararlı yazılıma ait **md5**, **sha1** ve **sha256** özet değerleri (hash), dosyanın eşsiz dijital parmak izleridir. Olay müdahale (Incident Response) süreçlerinde bu hash değerleri, dosyanın daha önce global Threat Intelligence (Tehdit İstihbaratı) platformlarında (Örn: VirusTotal, AlienVault OTX) raporlanıp raporlanmadığını sorgulamak için **IoC (Indicator of Compromise - Uzlaşma Göstergesi)** olarak kullanılır. Gelişmiş analizlerde, zararlı kendini değiştirse bile (polymorphism), bu hash değerlerinin takibi kritik bir başlangıç noktasıdır.

Çıktıdaki **analysis: static** satırı, dosyanın bilfiil çalıştırılmadığını (execute edilmediğini), kural tabanlı bir eşleştirme yapıldığını doğrular. **os: windows, format: pe** (Portable Executable) ve **arch: i386** satırları ise bir araya geldiğinde çok net bir mimariyi işaret eder: Karşımızdaki dosya, 32-bit Windows sistemler için derlenmiş çalıştırılabilir bir formattır (.exe , .dll , vb.).

ÖNEMLİ: Eğer bu aşamada mimariyi (i386) doğru tespit edemezsek, bir sonraki aşama olan tersine mühendislik safhasında (örneğin IDA Pro veya x64dbg kullanırken) yanlış decompiler veya debugger mimarisini seçeriz ve analizimiz tamamen çöker.

MITRE ATT&CK Çerçevesi ile Tehdit Modellemesi

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge), saldırganların bir siber saldırının her aşamasında kullandığı taktik ve teknikleri belgeleyen, global olarak kabul görmüş bir bilgi havuzu ve stratejik bir başvuru kaynağıdır. CAPA, bulduğu her davranışı bu standart çerçeveye oturtarak, savunan tarafa (Blue Team) "Düşman şu an hangi aşamada ve amacı ne?" sorusunun cevabını verir.

CAPA'nın MITRE çıktı formatı şu katı sözdizimini (syntax) kullanır: **ATT&CK Tactic :: ATT&CK Technique :: ATT&CK Sub-Technique [Technique Identifier.Sub-Technique Identifier]**

Bunu bir örnekle somutlaştıralım: **Defense Evasion::Obfuscated Files or Information::Indicator Removal from Tools [T1027.005]** Burada ana hedef/taktik **Defense Evasion** (Savunmadan Kaçınma) olarak belirlenmiştir. Bu amaca ulaşmak için kullanılan genel teknik **Obfuscated Files or Information** (Dosyaları veya Bilgiyi Gizleme) iken, bunun spesifik alt tekniği **Indicator Removal from Tools** (Araçlardan Gösterge Kaldırma) olarak raporlanmıştır. Köşeli parantez içindeki **T1027.005** ise bu davranışın global ID numarasıdır.

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Obfuscated Files or Information [T1027] Obfuscated Files or Information::Indicator Removal from Tools [T1027.005] Virtualization/Sandbox Evasion::System Checks [T1497.001]
DISCOVERY	File and Directory Discovery [T1083]
EXECUTION	Command and Scripting Interpreter::PowerShell [T1059.001] Shared Modules [T1129]
IMPACT	Resource Hijacking [T1496]
PERSISTENCE	Scheduled Task/Job::At [T1053.002] Scheduled Task/Job::Scheduled Task [T1053.005]

Terminal çıktısını incelediğimizde, **cryptbot.bin** isimli zararlının oldukça gürültülü ve agresif bir yapıya sahip olduğunu görüyoruz. **DEFENSE EVASION** taktiği altında listelenen **Virtualization/Sandbox Evasion::System Checks [T1497.001]** tekniği, zararlının analiz edildiğini anladığı an (bir sanal makine veya sandbox içinde olduğunu tespit ettiğinde) kendi çalışmasını durdurma yeteneğine sahip olduğunu kanıtlar.

EXECUTION aşamasında sistemin yerleşik yeteneklerini (Living off the Land) kullanarak **PowerShell** [T1059.001] süreçleri başlattığını; **PERSISTENCE** (Kalıcılık) sağlamak için ise Windows Görev Zamanlayıcısı'nı (Scheduled Tasks - [T1053.005]) kullanarak sistemi yeniden başlatsanız bile hayatta kalmayı hedeflediğini anlıyoruz. En can alıcı nokta ise **IMPACT** (Etki) başlığı altındaki Resource Hijacking [T1496] bulgusudur; bu, dosya adından da (cryptbot) anlaşılacağı üzere, hedefin sistem kaynaklarını sömürerek gizlice kripto para madenciliği (cryptojacking) yapmak olduğunu işaret eder.

MAEC (Malware Attribute Enumeration and Characterization) Sınıflandırması

MAEC, kötü amaçlı yazılımların karmaşık ayrıntılarını, davranışlarını ve diğer zararlılarla olan bağlantılarını standartlaştırmak için tasarlanmış özel bir dildir. Bir zararlıyı sadece "truva atı" veya "virüs" gibi basit kavramlarla değil, spesifik yetenekleriyle kodlar.

MAEC Category	MAEC Value
malware-category	launcher

CAPA, bu analizde **cryptbot.bin** dosyasını MAEC kategorisinde **Launcher** (Başlatıcı/Fırlatıcı) olarak etiketlemiştir. Sektörel terminolojide bu etiket, dosyanın bir nihai zararlı olmadığını, daha büyük bir operasyonun ilk ayağı olduğunu gösterir.

DİKKAT: Bir dosya **Launcher** olarak sınıflandırıldığında, bu dosyanın arka planda kendi içine gömülü (embedded) olan veya internetten sonradan çekeceği ikincil yükleri (Payloads) diske yazma (Dropping), bir C2 (Command and Control) sunucusuna bağlanma veya sistemde bir arka kapı (backdoor) açarak kalıcılık (persistence) mekanizmalarını tetikleme eğiliminde olduğunu öngörmeliyiz.

Eğer analiz edilen dosya MAEC tarafından **Downloader** (İndirici) olarak etiketlenseydi, analiz stratejimiz tamamen değişmek zorunda kalacaktı. Downloader'lar kendi içlerinde zararlı kod barındırmazlar; bunun yerine internete bağlanıp (örneğin bir Pastebin adresinden veya kompromize edilmiş bir WordPress sitesinden) yapılandırma dosyalarını veya asıl zararlıyı (Secondary Stages) RAM'e veya diske çekerler. Böyle bir senaryoda (B Planı), statik analizden ziyade dinamik analize ve ağ trafiği incelemesine (Wireshark/PCAP analizi) ağırlık vermemiz gerekirdi.

CAPA Sonuçlarının İncelenmesi Bölüm 2: Malware Behavior Catalogue (MBC)

CAPA aracının temel amacı, bir zararlı yazılımın (malware) yeteneklerini tespit etmek ve bunları standartlaştırılmış bir formatta sunmaktır. Bu analiz sürecinde karşımıza çıkan en kritik yapılardan biri **Malware Behavior Catalogue (MBC)**, yani Zararlı Yazılım Davranış Kataloğu'dur. MBC, zararlı yazılımın ne yapmaya çalıştığını (Objective) ve bunu nasıl

yaptığını (Behavior/Method) anlamlandırmak için oluşturulmuş sektörel bir referans kaynağıdır.

Aşağıdaki tablo, **CAPA** aracının analiz sonucunda ürettiği ham terminal çıktısının saf ve yapılandırılmış halidir. Bu tablo, zararlının hangi amaçlar doğrultusunda hangi teknikleri kullandığını gösteren nihai bir sistem anatomisidir.

CAPA: Temel MBC Analiz Tablosu

MBC Objective (Hedef)	MBC Behavior (Davranış ve Alt-Teknikler)
ANTI-BEHAVIORAL ANALYSIS	Virtual Machine Detection [B0009]
ANTI-STATIC ANALYSIS	Executable Code Obfuscation::Argument Obfuscation [B0032.020]
ANTI-STATIC ANALYSIS	Executable Code Obfuscation::Stack Strings [B0032.017]
COMMUNICATION	HTTP Communication [C0002]
COMMUNICATION	HTTP Communication::Read Header [C0002.014]
DATA	Check String [C0019]
DATA	Encode Data::Base64 [C0026.001]
DATA	Encode Data::XOR [C0026.002]
DEFENSE EVASION	Obfuscated Files or Information::Encoding-Standard Algorithm [E1027.m02]
DISCOVERY	File and Directory Discovery [E1083]
EXECUTION	Command and Scripting Interpreter [E1059]
FILE SYSTEM	Create Directory [C0046]
FILE SYSTEM	Delete File [C0047]
FILE SYSTEM	Read File [C0051]
FILE SYSTEM	Writes File [C0052]
MEMORY	Allocate Memory [C0007]
PROCESS	Create Process [C0017]

Malware Behavior Catalogue (MBC) ve MITRE ATT&CK İlişkisi

MBC, zararlı yazılımların hedeflerini ve davranışlarını etiketlemek (labelling), benzerlik analizi yapmak ve standartlaştırılmış raporlar sunmak için tasarlanmıştır. Güvenlik sektöründe **MITRE ATT&CK** çerçevesi referans alınsa da, ATT&CK daha çok ağ ve sistem genelindeki saldırgan davranışlarına (TTPs) odaklanır. **MBC ise doğrudan zararlı yazılımın (binary) kendi içindeki mikroskobik davranışlarına odaklanır.**

ÖNEMLİ: MBC, ATT&CK yöntemleriyle bağlantı kurar ancak onu kopyalamaz. MBC'nin amacı ATT&CK sayfalarındaki bilgiyi *tamamlamaktır*. Bir davranışın MBC adı, karşılık gelen ATT&CK tekniğiyle birebir aynı olmak zorunda değildir. Bu yapı, analistin zararlıyı karakterize ederken koda özel bir terminoloji kullanmasını sağlar.

CAPA Çıktı Formatının Anatomisi

CAPA sonuçlarında veriler hiyerarşik olarak sunulur. Logları okurken analistin bu hiyerarşiyi doğru çözümlemesi hayati önem taşır.

Format Tipi	Örnek Çıktı	Hiyerarşik Çözümleme
Format 1 (Tam Yapı)	ANTI-STATIC ANALYSIS::Executable Code Obfuscation::Argument Obfuscation [B0032.020]	OBJECTIVE: Anti-static Analysis BEHAVIOR: Executable Code Obfuscation METHOD: Argument Obfuscation IDENTIFIER: B0032.020
Format 2 (Temel Yapı)	COMMUNICATION::HTTP Communication:: [C0002]	OBJECTIVE: COMMUNICATION BEHAVIOR: HTTP Communication IDENTIFIER: C0002

DİKKAT: İki format arasındaki tek fark, birinci yapının **METHOD (Alt-teknik)** içermesidir. Bir metod, o davranışın spesifik olarak *hangi algoritma veya teknikle* (örneğin Base64 veya XOR) uygulandığını gösterir.

Objective (Hedef) ve Micro-Objective (Mikro-Hedef)

Objective (Hedef): Zararlı yazılımın nihai amacını tanımlar. Bunlar büyük oranda MITRE ATT&CK taktiklerine dayanır, ancak MBC zararlı analizi için **Anti-Behavioral** ve **Anti-Static**

Analysis gibi özel kategoriler ekler.

Objective (Hedef)	Teknik Açıklama ve Arka Plan
Anti-Behavioral Analysis	Zararlı yazılım, bir Sandbox (dinamik analiz ortamı) veya Debugger (hata ayıklayıcı) içinde olup olmadığını kontrol eder. Amacı, analiz edildiğini anladığında zararlı davranışlarını durdurarak tespitten kaçınmaktır.
Anti-Static Analysis	Kodu okumaya çalışan bir analisti (Reverse Engineer) engellemek için kod karmaşılaştırılır (Obfuscation), paketlenir (Packing) veya şifrelenir.
Collection	Hedef makineden veya ağdan hassas verileri, dosyaları ve yapılandırmaları belirleme ve toplama (Recon & Data Harvesting) sürecidir.
Command and Control	Komuta ve Kontrol (C2). Zararlı yazılımın, saldırganın sunucusuyla veya bir P2P ağıyla kurduğu iletişim kanalıdır. Bu kanal üzerinden komut alır veya veri sızdırır.
Credential Access	İşletim sistemi belleğinden (Örn: LSASS), tarayıcılardan veya dosyalardan kullanıcı adı, parola ve hash değerlerini (Kimlik Bilgileri) çalma girişimidir.
Defense Evasion	Sistemdeki Antivirüs, EDR, Firewall gibi güvenlik mekanizmalarını atlatma, devre dışı bırakma veya gizlenme çabasıdır.
Discovery	Sistemde keşif yapma sürecidir. İşletim sistemi mimarisi, ağ yapılandırması, kurulu yazılımlar ve donanım hakkında bilgi toplanır.
Execution	Zararlı makine kodunun (Shellcode) veya izinsiz komutların hedef sistemde çalıştırılması eylemidir.
Exfiltration	Toplanan hassas verilerin, ağ kısıtlamalarını aşarak saldırganın kontrolündeki dış bir sunucuya (C2) sızdırılmasıdır.
Impact	Sistem bütünlüğünü bozma, fidye yazılımı (Ransomware) gibi dosyaları şifreleme veya sistemi kullanılamaz hale getirme (Wiper/DoS) eylemlerini kapsar.
Lateral Movement	İlk sömürülen makineden ağdaki diğer sunuculara veya cihazlara sıçrama (Yanal Hareket) işlemidir. WMI, SMB, RDP gibi protokoller kullanılır.
Persistence	Zararlı yazılımın, sistem yeniden başlatılsa bile (Reboot) aktif kalmasını sağlayan kalıcılık (Kayıt defteri, zamanlanmış görevler vb.) mekanizmalarıdır.
Privilege Escalation	Düşük yetkili bir kullanıcının veya sürecin, Admin, SYSTEM veya root gibi yüksek yetkilere erişmesi (Yetki Yükseltme) işlemidir.

Micro-Objective (Mikro-Hedef): Kendi başlarına doğrudan zararlı olmayan ancak zararlı yazılımlar tarafından istismar edilen temel işletim sistemi etkileşimlerini tanımlar. CAPA çıktısında genel "Objective" sütunu altında listelenirler.

Micro-Objective	Teknik Kapsam
PROCESS	İşlem oluşturma (Create Process), Thread (İş parçacığı) bağlamı ayarlama, işlem sonlandırma ve Mutex (Tekil çalışma kilidi) kontrolü gibi davranışları kapsar.
MEMORY	Bellek tahsis etme (Allocate Memory), bellek korumasını değiştirme (Örn: RWX izinleri ayarlama) ve bellek temizleme işlemleridir. Paket açma (Unpacking) süreçlerinin temelidir.
COMMUNICATION	DNS, FTP, HTTP, ICMP, SMTP gibi standart ağ trafiği oluşturma eylemleridir. C2 iletişimi için paravan olarak kullanılır.
DATA	Metin dizilerini (Strings) kontrol etme, veriyi sıkıştırma, şifreleme (Encoding) ve çözme (Decoding) eylemleridir.

Behavior (Davranış) ve Micro-Behavior (Mikro-Davranış)

Davranışlar, hedefe ulaşmak için atılan somut adımlardır. Aşağıdaki tablolar, CAPA'nın temel davranışları ve mikro-davranışları nasıl sınıflandırdığını göstermektedir.

Temel Behaviors (Davranışlar):

Objective	Behavior	Identifier	Teknik Açıklama
ANTI-BEHAVIORAL ANALYSIS	Virtual Machine Detection	B0009	Zararlı yazılım, sistemdeki donanım eserlerini (MAC adresleri, sürücüler) inceleyerek sanal bir makinede (VMware, VirtualBox) çalışıp çalışmadığını denetler.
ANTI-STATIC ANALYSIS	Executable Code Obfuscation	B0032	Analizi zorlaştırmak için çalıştırılabilir dosyanın (PE/ELF) veri veya metin bölümleri kasıtlı olarak karartılır.
EXECUTION	Command and Scripting Interpreter	E1059	Zararlı, işlemleri yürütmek için işletim sisteminin yerleşik yorumlayıcılarını (cmd.exe, PowerShell, Bash) veya Python, JavaScript gibi dilleri sömürür.
DISCOVERY	File and Directory Discovery	E1083	Belirli kritik dosyaları (Örn: yapılandırma veya şifre dosyaları) bulmak için dosya sisteminde izinleri ve dosyaları listeler (Enumeration).
ANTI-STATIC ANALYSIS, DEFENSE EVASION	Obfuscated Files or Information	E1027	Dosyaları veya bilgileri şifreleyerek ya da kodlayarak gizler. Ayrıca zararlı yazılım kendi kendini paketleyerek (packing) statik imzaları atlatabilir.

Micro-Behaviors (Mikro-Davranışlar / Düşük Seviye Eylemler):

Bunlar API çağrıları veya sistem fonksiyonlarıyla doğrudan bağlantılı "düşük seviyeli" eylemlerdir.

Micro-Objective	Micro-Behavior	Identifier	Teknik Açıklama
MEMORY	Allocate Memory	C0007	Zararlı yazılım, kendi Payload'unu (zararlı kod bloğunu) hafızaya açmak (unpack) ve çalıştırmak için işletim sisteminden bellek tahsis eder.
PROCESS	Create Process	C0017	WMI veya Shellcode aracılığıyla yeni bir işlem başlatır. Süreci askıya alarak (Suspended Process) içine zararlı kod enjekte edebilir (Process Hollowing).
COMMUNICATION	HTTP Communication	C0002	Dış dünyayla iletişim kurmak için HTTP protokolünü başlatır. (Genellikle C2 Callback için).
DATA	Check String	C0019	Bellekteki veya dosyadaki metinlerin spesifik özelliklerini (ASCII yapısı, kredi kartı formatı, uzunluk vb.) denetler.
DATA	Encode Data	C0026	Veriyi düz metin olarak bırakmaz, Base64 veya XOR gibi algoritmalarla kodlayarak gizler.
FILE SYSTEM	Create Directory	C0046	Sisteme kendi dosyalarını veya çaldığı verileri depolamak için yeni bir dizin açar.
FILE SYSTEM	Delete File	C0047	İzini kaybettirmek veya sistem bütünlüğüne zarar vermek için dosyaları siler (Anti-Forensics).
FILE SYSTEM	Read File	C0051	Hassas verileri veya yapılandırmaları toplamak için sistemdeki dosyaları okur.
FILE SYSTEM	Writes File	C0052	Sisteme yeni bir zararlı modül (Dropper) veya toplanan verilerin bulunduğu bir arşiv yazar.

Methods (Yöntemler / Alt-Teknikler)

Yöntemler, bir davranışın *tam olarak hangi teknik kullanılarak* uygulandığını gösteren en ince ayrıntılardır.

Behavior (Davranış)	Method (Yöntem)	Identifier	Teknik Açıklama
Executable Code Obfuscation	Argument Obfuscation	B0032.020	API çağrılarına gönderilen argümanlar kod içinde açıkça bulunmaz; analiz sürecini zorlaştırmak için çalışma zamanında (Runtime) hesaplanır.
Executable Code Obfuscation	Stack Strings	B0032.017	strings analizini atlatmak için kritik metinler, Stack (yığın) belleği üzerinde her kullanımda karakter karakter oluşturulur ve hemen ardından silinir.
HTTP Communication	Read Header	C0002.014	İletişim sırasında HTTP başlıklarını (Header) okuyarak C2 sunucusundan gelen özel komutları veya durum kodlarını işler.
Encode Data	Base64	C0026.001	Veriyi kodlamak için Base64 standart algoritmasını kullanır.
Encode Data	XOR	C0026.002	Veri sızdırma veya gizleme için XOR mantıksal operatörünü şifreleme algoritması olarak kullanır.
Obfuscated Files or Information	Encoding-Standard Algorithm	E1027.m02	Zararlı modülleri, konfigürasyonları veya ağ trafiğini gizlemek için bilinen standart algoritmaları uygular.

Nihai Analiz ve Mantıksal Yorumlama (Örnek Vaka)

Tüm bu tablo ve veri yığınlarının pratik hayattaki (veya bir sınavdaki) analiz mantığı şu şekildedir. Aşağıdaki tekil log örneğini inceleyelim:

MBC Objective	MBC Behavior
DATA	Encode Data::Base64 [C0026.001]

Analitik Yorum (Mantık Zinciri):

- Label: MBC Objective / Value: DATA:** Analist olarak ilk olarak zararlının sistemdeki verilerle (metinler, yapılandırmalar vb.) manipülatif bir şekilde etkileşime girdiğini anlıyoruz.
- Label: MBC Behavior / Value: Encode Data:** Zararlı yazılım verileri düz (Cleartext) olarak işlemiyor veya göndermiyor; onları kodlayarak okunabilirliğini bozma yeteneğine sahip.

3. **Label: Method / Value: Base64:** Bu kodlama davranışı rastgele bir şifrelemeyle değil, spesifik olarak **Base64** metodolojisiyle gerçekleştiriliyor.
4. **Label: Identifier / Value: C0026.001:** Bu eylem sektörel çapta tag'lenmiştir (etiketlenmiştir).

Uygulama Sonucu: Bu çıktıya bakarak bir analist raporunda *"Dosya Base64 kullanıyor"* demez. Bunun yerine: *"Zararlı yazılımın (DATA hedefi doğrultusunda), ağ trafiğindeki (Payload) anormallikleri veya sistemdeki konfigürasyon dosyalarını gizlemek amacıyla veriyi Base64 metodolojisiyle (C0026.001) kodlama (Encode Data) yeteneğine sahip olduğunu tespit ediyoruz."* şeklinde tam teşekküllü bir çıkarım yapar.

CAPA Sonuçlarının İncelenmesi Bölüm 3: Namespaces (İsim Uzayları)

CAPA analiz motoru, zararlı yazılımın (malware) davranışlarını tespit ettikten sonra bunları analiste sunarken rastgele bir liste vermez. Çıktıyı mantıksal bir çerçeveye oturtmak için **Capability** (Yetenek/Kural Adı) ve **Namespace** (İsim Uzayı) olmak üzere iki temel kavram kullanır. Bu bölüm, zararlı yazılımın *ne yapabildiğini* (Capability) ve bu yeteneğin sistemin *hangi alanında* (Namespace) konumlandığını anlamak üzerine inşa edilmiştir.

Aşağıdaki tablo, CAPA'nın terminalde ürettiği ham çıktıdaki verilerin Obsidian ve benzeri bilgi yönetimi (PKM) sistemleri için yapılandırılmış ve temizlenmiş halidir. Bu tablo, bir zararlı yazılımın sistemdeki hareket alanını gösteren topolojik bir haritadır.

CAPA: Capability ve Namespace Analiz Tablosu

Capability (Yetenek / Tetiklenen Kural)	Namespace (İsim Uzayı / Davranışın Kategorisi)
reference anti-VM strings	anti-analysis/anti-vm/vm-detection
reference anti-VM strings targeting VMWare	anti-analysis/anti-vm/vm-detection
reference anti-VM strings targeting VirtualBox	anti-analysis/anti-vm/vm-detection
contain obfuscated stackstrings (2 matches)	anti-analysis/obfuscation/string/stackstring
reference HTTP User-Agent string	communication/http
check HTTP status code	communication/http/client
reference Base64 string	data-manipulation/encoding/base64
encode data using XOR	data-manipulation/encoding/xor
contain a thread local storage (.tls) section	executable/pe/section/tls
get common file path	host-interaction/file-system

Capability (Yetenek / Tetiklenen Kural)	Namespace (İsim Uzayı / Davranışın Kategorisi)
create directory	host-interaction/file-system/create
delete file	host-interaction/file-system/delete
read file on Windows (4 matches)	host-interaction/file-system/read
write file on Windows (5 matches)	host-interaction/file-system/write
get thread local storage value	host-interaction/process
create process on Windows	host-interaction/process/create
allocate or change RWX memory	host-interaction/process/inject
reference cryptocurrency strings	impact/cryptocurrency
link function at runtime on Windows (5 matches)	linking/runtime-linking
parse PE header (4 matches)	load-code/pe
resolve function by parsing PE exports (186 matches)	load-code/pe
run PowerShell expression	load-code/powershell/
schedule task via at	persistence/scheduled-tasks
schedule task via schtasks	persistence/scheduled-tasks

Namespace Formatının Anatomisi ve Arka Plan Mantığı

CAPA çıktısındaki her bir satır, arka planda çalışan spesifik bir yara kuralının veya algoritmanın sonucudur. Bu sonuçların hiyerarşisi şu formattadır:

Format: Capability (Rule Name) :: TLN (Top-Level Namespace) / Namespace

Örnek bir log satırını `anti-analysis/anti-vm/vm-detection` parçalayalım:

- **Capability (Yetenek):** `reference anti-VM strings`. Bu, zararlı yazılımın gerçekleştirdiği spesifik eylemdir (Sanal makine tespitine yönelik metinler içeriyor).
- **TLN (Top-Level Namespace):** `anti-analysis`. Bu, en üst düzey kategoridir. Zararlıının bu eylemi yapmaktaki *ana motivasyonunu* belirtir (Analiz edilmekten kaçınmak).
- **Namespace (İsim Uzayı):** `anti-vm/vm-detection`. Bu, davranışın TLN altındaki spesifik teknik dalıdır.

ÖNEMLİ: CAPA'nın kalbinde **YAML** dosyaları yatar. Gördüğümüz her bir *Capability*, aslında arka planda yazılmış bir `.yaml` kural dosyasının tetiklenmesiyle ortaya çıkar. Namespaces, bu binlerce YAML dosyasını mantıksal klasörler halinde gruplamaya yarar.

Top-Level Namespace (TLN) Mimarisi

Bir analiz raporu hazırlarken, TLN'leri bilmek zararlı yazılımın genel profilini (Örn: "Bu bir fidye yazılımı mı yoksa veri sızdıran bir casus yazılım mı?") saniyeler içinde çıkarmanızı sağlar.

Top-Level Namespace (TLN)	Analitik Derinlik ve Teknik Açıklama
anti-analysis	Zararlıının Sandbox, Debugger (hata ayıklayıcı) veya analist ortamlarını tespit edip atlatmaya yönelik kuralları barındırır. (Obfuscation, Packing, Anti-Debugging).
collection	Veri sızdırma (Exfiltration) öncesinde, kurbandan klavye vuruşları, ekran görüntüleri veya hassas dosyaların toplanması (Data Gathering) sürecini içerir.
communication	Ağ etkileşimi. C2 (Command & Control) sunucusuyla kurulan bağlantılar, veri iletimi ve alınan komutların (Callback) analiz edildiği gruptur.
compiler	Çalıştırılabilir dosyanın (PE/ELF) hangi derleyici (Compiler) ortamında inşa edildiğini tespit eden "imzaları" içerir. Bu, zararlıyı yazan grubun geliştirme ortamı (Toolchain) hakkında istihbarat sağlar.
data-manipulation	Veri dönüştürme işlemleri. Zararlı yazılımın konfigürasyon dosyalarını şifrelemesi (String Encryption) veya sızdırılacak verileri kodlaması (Örn: Base64, XOR) bu uzaydadır.
executable	Dosyanın kendi statik yapısıyla ilgili özellikleri (Örn: PE bölümleri, .tls section, Debug bilgileri) inceler.
host-interaction	İşletim sistemiyle olan doğrudan etkileşimdir. Dosya sistemine erişim (okuma/yazma/silme), kayıt defteri (Registry) manipülasyonları veya yeni süreçler (Process) başlatma işlemlerini kapsar.
impact	Zararlıının sistemde yaratacağı <i>nihai hasar</i> potansiyelidir. Fidye yazılımının dosyaları şifrelemesi, bir Wiper'ın diskleri silmesi veya kripto madenciliği (Cryptojacking) faaliyetleri buradadır.
linking	Programın çalışırken dış kütüphanelere (Örn: OpenSSL, Zlib) dinamik olarak bağlanması veya kod yüklemesi sürecidir. <i>Neden Kritik?</i> Zararlıının dış kütüphanelere bağımlılığı, onun kapasitesini gösterir. Ayrıca, bağlanan zafiyetli bir kütüphane analiz sırasında defansif ekipler tarafından istismar edilebilir.
load-code	Belleğe dinamik olarak yetkisiz kod enjekte edilmesi (Runtime Code Injection) davranışlarını inceler.
malware-family	APT (Gelişmiş Sürekli Tehdit) gruplarına veya bilinen spesifik zararlı yazılım ailelerine ait karakteristik özellikleri (imzaları) eşleştirir.

Top-Level Namespace (TLN)	Analitik Derinlik ve Teknik Açıklama
persistence	Kalıcılık. Sistem yeniden başlatıldığında (Reboot) zararlının hayatta kalmasını sağlayan mekanizmalardır (Örn: Zamanlanmış görevler - Scheduled Tasks, AutoRun kayıtları).
runtime	Zararlının çalışması için gereken temel dili veya platformu (Örn: .NET, Java, Go) tespit eder.
internal / lib / nursery	Bunlar CAPA'nın kendi iç motorunun kullandığı, analiste doğrudan tehdit istihbaratı sağlamayan, yapı taşı ve test aşamasındaki kuralları içeren teknik arka plan uzaylarıdır.
targeting	Hedefli saldırılar. Örneğin doğrudan ATM cihazlarını sömürmek için yazılmış spesifik kuralları barındırır.

Namespace Hiyerarşisi ve YAML Bağlantısı (Mantık Zinciri)

CAPA'nın mimarisini tam olarak kavramak için kuralların (Rules) nasıl gruplandığına, yani klasörleme mantığına bakmalıyız. Analizde gördüğümüz "Davranış", aslında bir YAML dosyasının tespittir.

Örnek olarak `Anti-Analysis` (TLN) uzayını ele alalım.

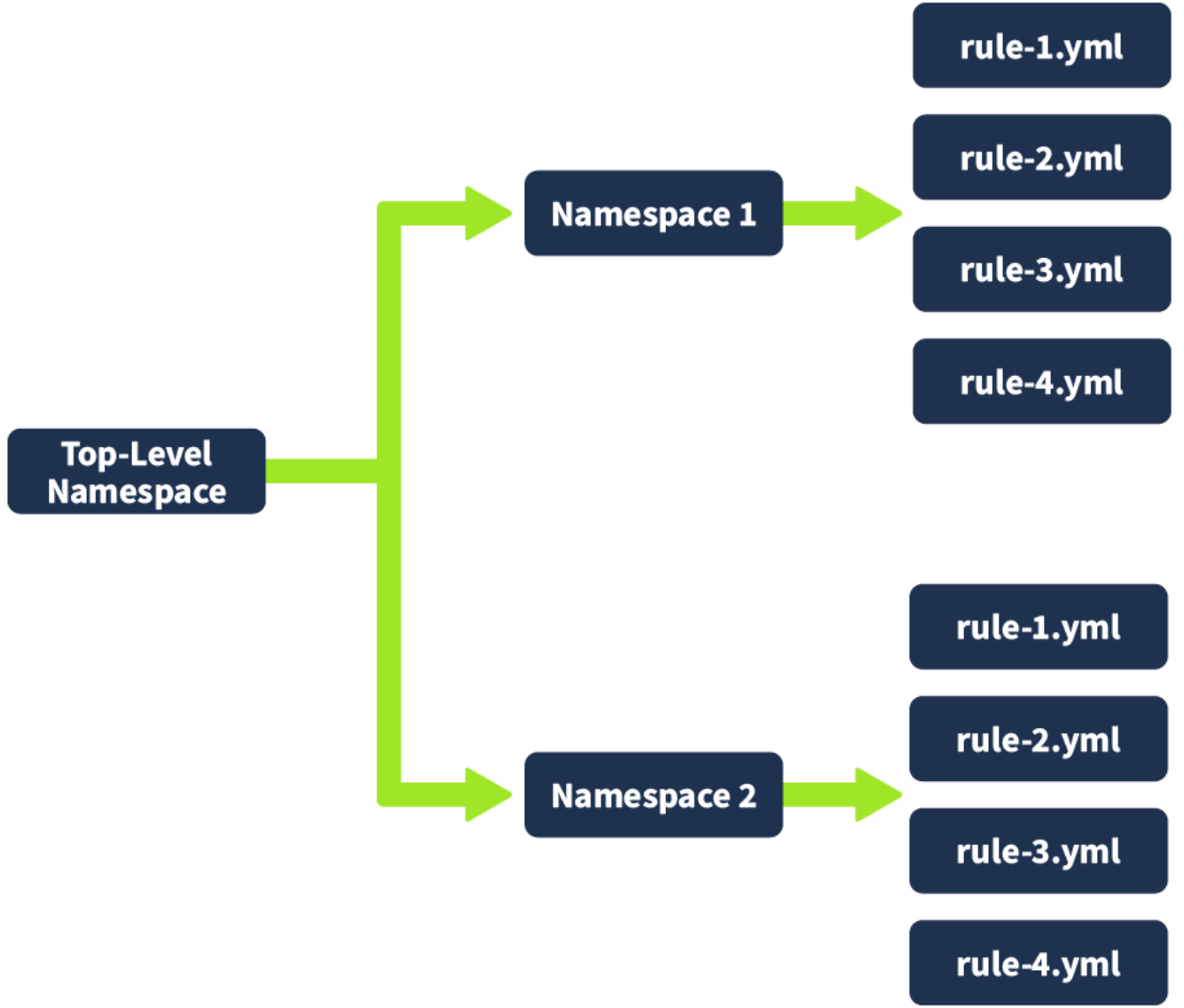
1. Dal: Sanal Makine Tespiti (anti-vm/vm-detection)

Eğer zararlı yazılım sistemde VirtualBox veya VMware arıyorsa, `Anti-Analysis` klasörü altındaki `anti-vm/vm-detection` isim uzayı devreye girer. Bu uzayın içinde, spesifik tehditleri arayan şu YAML kuralları çalışır:

- `reference-anti-vm-strings-targeting-virtualbox.yml`
- `reference-anti-vm-strings-targeting-virtualpc.yml`

Analitik Yorum: Zararlı, sanallaştırma araçlarına ait spesifik kayıt defteri anahtarlarını veya sürücü dosyalarını (artifacts) arıyordur. CAPA bu statik dizeleri bulduğunda ilgili YAML kuralı "Eşleşti (Match)" uyarısı verir.

2. Dal: Kod Karmaşılaştırma (obfuscation)



Aynı Anti-Analysis TLN'si altında, zararlının kendini gizleme çabalarını tespit eden obfuscation isim uzayı bulunur. Bu uzayın içinde de kendi spesifik YAML kuralları mevcuttur:

- obfuscated-with-dotfuscator.yml
- obfuscated-with-smartassembly.yml

Analitik Yorum: Burada zararlı yazılım sanal makine aramıyor; ancak .NET tabanlı kodunu analiz araçlarından gizlemek için Dotfuscator gibi ticari/zararlı araçlarla şifreliyor (String Encryption) veya paketliyor (Packing).

DİKKAT: Bir zararlı yazılım analizinde Anti-Analysis TLN'si altında yoğun kurallar (Capability) tetikleniyorsa, analist olarak bu zararlının basit bir script olmadığını, analizi zorlaştırmak üzere profesyonelce (örneğin bir APT grubu tarafından) dizayn edildiğini ve dinamik analiz ortamlarınızı (Sandbox) atlatabileceğini varsaymalısınız. Bu durumda kodu doğrudan çalıştırmak yerine, önce hafıza analizine (Memory Forensics) veya tersine mühendisliğe (Reverse Engineering) odaklanmak B planı olmalıdır.

CAPA Sonuçlarının İncelenmesi Bölüm 4: Capability (Yetenekler) ve YAML Mimarisi

Önceki bölümlerde zararlı yazılımın nihai hedeflerini (Objective) ve bu hedeflerin sistemdeki topolojik haritasını (Namespace) inceledik. Bu bölümde ise **CAPA** analiz motorunun kalbine inerek, zararlı yazılımın tespit edilen somut eylemlerini temsil eden **Capability** (Yetenek) kavramını ve bu tespitlerin arka plandaki kural motoruyla (YAML dosyaları) nasıl eşleştiğini parçalayacağız.

Bir analist olarak CAPA çıktısındaki bir Capability satırını okuduğunuzda, aslında doğrudan bir tersine mühendislik (Reverse Engineering) kuralının tetiklendiğini görürsünüz. Aşağıdaki tablo, bu yeteneklerin en üst seviye kategoriden (TLN) spesifik kural dosyasına (YAML) kadar uzanan tam hiyerarşik haritasını göstermektedir.

CAPA: Capability, Namespace ve YAML Kural Eşleşme Tablosu

Capability (Tespit Edilen Yetenek)	Top-Level Namespace (TLN)	Namespace (İsim Uzayı)	Rule YAML File (Tetiklenen Kural Dosyası)
reference anti-VM strings	Anti-Analysis	anti-vm/vm-detection	reference-anti-vm-strings.yml
reference anti-VM strings targeting VMWare	Anti-Analysis	anti-vm/vm-detection	reference-anti-vm-strings-targeting-vmware.yml
reference anti-VM strings targeting VirtualBox	Anti-Analysis	anti-vm/vm-detection	reference-anti-vm-strings-targeting-virtualbox.yml
reference HTTP User-Agent string	Communication	http/client	reference-http-user-agent-string.yml
check HTTP status code	Communication	http	check-http-status-code.yml
reference Base64 string	Data Manipulation	encoding/base64	reference-base64-string.yml
encode data using XOR	Data Manipulation	encoding/xor	encode-data-using-xor.yml
contain a thread local storage (.tls) section	Executable	pe/section/tls	contain-a-thread-local-storage-tls-section.yml
get common file path	Host-Interaction	file-system	get-common-file-path.yml
create directory	Host-Interaction	file-system/create	create-directory.yml
delete file	Host-Interaction	file-system/delete	delete-file.yml

Capability (Tespit Edilen Yetenek)	Top-Level Namespace (TLN)	Namespace (İsim Uzayı)	Rule YAML File (Tetiklenen Kural Dosyası)
read file on Windows	Host-Interaction	file-system/read	read-file-on-windows.yml
write file on Windows	Host-Interaction	file-system/write	write-file-on-windows.yml
get thread local storage value	Host-Interaction	process	get-thread-local-storage-value.yml
allocate or change RWX memory	Host-Interaction	process/inject	allocate-or-change-rwx-memory.yml
create process on Windows	Host-Interaction	process/create	create-process-on-windows.yml
reference cryptocurrency strings	Impact (<i>İstisna: Nursery</i>)	impact/cryptocurrency	reference-cryptocurrency-strings.yml
link function at runtime on Windows	Linking	runtime-linking	link-function-at-runtime-on-windows.yml
parse PE header	load-code	load-code/pe	parse-pe-header.yml
resolve function by parsing PE exports	load-code	load-code/pe	resolve-function-by-parsing-pe-exports.yml
run PowerShell expression	load-code	load-code/powershell	run-powershell-expression.yml
schedule task via at	persistence	scheduled-tasks	schedule-task-via-at.yml
schedule task via schtasks	persistence	scheduled-tasks	schedule-task-via-schtasks.yml

Capability ve YAML Arasındaki Simetrik Bağlantı

Bu tabloyu dikkatle incelediğinizde kritik bir isimlendirme standardı (Naming Convention) fark edeceksiniz. Terminal çıktısında bir **Capability** (Yetenek) olarak gördüğünüz isim, aslında arka planda çalışan statik veya dinamik kural dosyasının (YAML) ta kendisidir.

Tek fark sözdizimseldir: **Capability** adındaki boşluklar, **YAML** dosyasında tire (-) işareti ile değiştirilir.

Örneğin; loglarda `schedule task via schtasks` şeklinde okuduğunuz bir yetenek, CAPA'nın veritabanındaki `schedule-task-via-schtasks.yml` kuralının o zararlı yazılım üzerinde eşleştiği (Match) anlamına gelir.

ÖNEMLİ: Bu simetri elit bir analist için neden önemlidir? Eğer CAPA bir dosyayı spesifik bir özellikyle etiketlediyseniz ve siz bu tespiti *yanlış pozitif (False Positive)* olabileceğinden şüpheleniyorsanız veya CAPA'nın tam olarak neyi aradığını öğrenmek istiyorsanız, doğrudan GitHub'daki veya lokalinizdeki CAPA kuralları dizinine gidip o YAML dosyasını açabilirsiniz. Dosyanın içinde API çağrıları, spesifik Assembly komutları veya tespit için kullanılan Byte dizileri açıkça yazılıdır.

Vakalar Üzerinden Mantıksal Çözümleme

Yukarıdaki tablodan birkaç kritik yeteneği çekerek arka plandaki sistem mühendisliği mantığını kuralım:

Vaka 1: Sanal Makine Tespiti (Anti-VM)

- **Capability:** `reference anti-vm strings`
- **Hiyerarşi:** Anti-Analysis (TLN) -> `anti-vm/vm-detection (Namespace)` -> `reference-anti-vm-strings.yml`
- **Teknik Derinlik:** Zararlı yazılımlar genellikle bir Sandbox ortamında izole edildiklerini anlamak için sistemde spesifik izler ararlar. CAPA bu kuralı tetiklediğinde, incelenen dosyanın (binary) içinde VMware veya VirtualBox'a ait spesifik kayıt defteri anahtarları (Registry Keys), MAC adres blokları (OUI) veya `vBoxTray.exe` gibi sürücü/araç isimlerinin (Strings) bulunduğunu anlıyoruz. Bu durum, zararlının analizden kaçınmak için pasif bir keşif aşamasında olduğunu kanıtlar.

Vaka 2: Windows Görev Zamanlayıcısı Üzerinden Kalıcılık

- **Capability:** `schedule task via schtasks`
- **Hiyerarşi:** persistence (TLN) -> `scheduled-tasks (Namespace)` -> `schedule-task-via-schtasks.yml`
- **Teknik Derinlik:** Bir zararlı yazılım sisteme ilk sızdığı anda (Initial Access), sistem yeniden başlatıldığında (Reboot) bağlantısını kaybetmemek zorundadır. Bu çıktı bize, zararlının Windows İşletim Sistemine ait meşru bir araç olan **`schtasks.exe`**'yi çağırarak (veya doğrudan COM nesneleri üzerinden Task Scheduler API'sini kullanarak) kendi kendini zamanlanmış bir görev olarak sisteme kaydettiğini gösterir. Bu net bir kalıcılık (Persistence) göstergesidir.

İstisnai Durumlar ve Nursery (Kuluçka) Uzayı

Her kural doğrudan mantıksal İsim Uzayı (Namespace) altında fiziksel olarak bulunmayabilir. Analiz sırasında klasör yapısını incelerken karşınıza çıkacak en önemli istisna **Nursery (Kuluçka)** uzayıdır.

Tablodaki `reference cryptocurrency strings` yeteneğini ele alalım. Mantıksal olarak bir kripto para cüzdanı aramak veya madencilik (Cryptojacking) yapmak, sistemde bir hasar veya kazanç yaratacağı için **Impact** isim uzayı altında olmalıdır. Ancak CAPA kural dizininde bu kuralı Impact klasöründe bulamazsınız; fiziksel olarak **Nursery** dizinindedir.

DİKKAT: Nursery uzayı, henüz tamamen cilalanmamış (unpolished), test aşamasında olan veya *yanlış pozitif (False Positive)* üretme ihtimali yüksek olan kuralların tutulduğu bir bekleme alanıdır. Bir analist, Nursery içindeki bir kuralın tetiklendiğini gördüğünde, bu yeteneği kesin bir olgu olarak rapora yazmadan önce bulguyu manuel olarak (Örn: IDA Pro veya x64dbg kullanarak) doğrulaması gerektiğini bilir.

Nihai Konsolide Analiz

Eğitim boyunca edindiğimiz tüm hiyerarşik bilgiyi tek bir çıktı üzerinde birleştirerek bir analist yorumu oluşturalım:

Capability	Namespace
reference Base64 string	data-manipulation/encoding/base64

Label (Etiket)	Value (Bulgu / Değer)	Teknik Açıklama ve Arka Plan
Capability	reference base64 string	Zararlı yazılım, sistemdeki verileri Base64 şemasını kullanarak kodlama (encode) yeteneğine sahiptir.
Top-Level Namespace	data-manipulation	Çalıştırılabilir dosya (executable) içindeki verilerin manipüle edilmesiyle ilgili davranışları yakalayan kural setlerini içerir. Bu alan, String Şifreleme (String Encryption) ve Veri Kodlama (Data Encoding) gibi eylemleri kapsayan zararlı yazılımın "veri dönüştürme" bileşenidir.
Namespace	encoding/base64	Bu spesifik isim uzayı, verileri özel olarak Base64 (ve bazen XOR) algoritmalarını kullanarak kodlamak ve çözmek için yazılmış alt kurallardan oluşur.
Rule YAML File Matched?	reference-base64-string.yml	Logda tetiklenen yeteneğin arka plandaki fiziksel dosyasıdır. ÖNEMLİ NOT: Capability (Yetenek) ismindeki boşlukların tire (-) karakteri ile değiştirilmesi, bize doğrudan GitHub veya kural dizinindeki hedef kural dosyasının adını verir.

CAPA: Zararlı Yazılım Analizinde İleri Seviye Kural Tespitleri ve Very Verbose (Çok Detaylı) Çıktı Yönetimi

Zararlı yazılımların (Malware) yeteneklerini analiz ederken yalnızca bir kuralın tetiklendiğini bilmek yeterli değildir; bir siber güvenlik uzmanı olarak o kuralın tam olarak **neden** ve **hangi koşullar altında** eşleştiğini kanıtlamak zorundayız. Bu derinlemesine analizi gerçekleştirmek

Compromise - Uzlaşma Göstergesi) değerlerini gözden kaçırmamıza neden olur. Bu veriyi terminal üzerinden okumak profesyonel bir yaklaşım değildir.

Büyük Çıktıların Otomatize Edilmesi ve JSON Formatına Çevrilmesi

Saf metin (TXT) dosyaları terminalde veya Notepad gibi araçlarda okunması zor, filtrelenebilir yapılarıdır. Analiz edilecek veriyi kolayca ayrıştırmak (Parsing) ve bir sonraki adımda kullanacağımız grafiksel arayüze (GUI) besleyebilmek için çıktıyı makine tarafından okunabilir bir format olan **.json**'a dönüştürmeliyiz.

Bunu yapmak için araca JSON çıktısı üretmesini söyleyen `-j` parametresini dahil ediyor ve sonucu bir dosyaya yönlendiriyoruz (`>`).

```
capa.bin -j -vv .\cryptbot.bin > cryptbot_vv.json
```

Çıktı yorumu ve teknik analiz:

```
loading : 100%|████████████████████| 485/485 [00:00<00:00, 1108.84  
rules/s]/ analyzing program...
```

Burada aracı hem JSON formatında (`-j`) hem de en yüksek detay seviyesinde (`-vv`) çalıştırdık. Çıktı, terminal ekranına basılmak yerine işletim sisteminin standart çıktı yönlendirme operatörü (`>`) sayesinde doğrudan **`cryptbot_vv.json`** dosyasına yazıldı.

ÖNEMLİ (B Planı): Eğer işletim sistemi izinleri (Permissions) veya izin kısıtlamaları nedeniyle bu yönlendirme işlemi başarısız olsaydı, "Access Denied" hatası alırdık. Böyle bir durumda B planımız komutu Administrator (Yönetici) yetkileriyle çalıştırılmış bir terminalde denemek veya çıktıyı herkesin yazma yetkisi olan geçici bir dizine (Örn: `>`
`$env:TEMP\cryptbot_vv.json`) yönlendirmek olmalıdır. Alternatif olarak, komut satırında JSON manipülasyonu için `jq` gibi araçlar kullanarak veriyi anlık olarak filtreleyebilirdik.

CAPA Web Explorer ile Görsel Mimari Analizi

Oluşturduğumuz devasa JSON dosyasını incelemenin en etkili yolu **CAPA Web Explorer** kullanmaktır. Bu araç, JSON verisini ayrıştırarak MITRE ATT&CK framework'üne uygun şekilde eşleşen yetenekleri (Capabilities) görselleştirir.

Sanal makinemizde yer alan çevrimdışı versiyonu (**`capa_web_explorer_offline.html`**) Chrome tarayıcısı üzerinden açıyoruz. Sol alt köşedeki **Upload from local** butonunu kullanarak **`C:\Users\Administrator\Desktop\capa`** yolundaki **`cryptbot_vv.json`** dosyasını sisteme yüklüyoruz.

cryptbot.bin SHA256: aa7b2d3d8ecb209a7e957e4bb86e0d11845c5b2d97a00a482be63b567ce4c	Windows · PE · i386 Static Analysis With Vivisect · capa v7.2.0 · 9/15/2024, 3:40:19 PM	31 RULES	22 NAMESPACES	9522 FUNCTIONS
☐ Show capabilities by function ☐ Show 6 distinct library rules ☐ Show namespace chart ☐ Show column filters				
Q Global search				
Rule ↑↓	Address	Namespace ↑	ATT&CK Technique ↑↓	Malware Behavior Catalog ↑↓
> reference anti-VM strings targeting VMWare		anti-analysis/anti-vm/vm-detection	Virtualization/Sandbox Evasion (T1497) ↳ System Checks (T1497.001)	Anti-Behavioral Analysis::Virtual Machine Detection [B0009]
> reference anti-VM strings targeting VirtualBox		anti-analysis/anti-vm/vm-detection	Virtualization/Sandbox Evasion (T1497) ↳ System Checks (T1497.001)	Anti-Behavioral Analysis::Virtual Machine Detection [B0009]
> reference anti-VM strings		anti-analysis/anti-vm/vm-detection	Virtualization/Sandbox Evasion (T1497) ↳ System Checks (T1497.001)	Anti-Behavioral Analysis::Virtual Machine Detection [B0009]

Bu grafiksel arayüz bize muazzam bir Global Search (Küresel Arama) yeteneği ve filtreleme opsiyonları sunar. Böylece üç bin satırlık text dosyasında aramak yerine, doğrudan belirli bir saldırı tekniğine (Örn: Defense Evasion) odaklanabiliriz.

Kural (Rule) Anatomisi ve Mantıksal Çözümleme

Zararlı yazılımın ne yapmaya çalıştığını anladık, şimdi CAPA'nın bunu "nasıl" anladığını inceleyeceğiz. Arka planda çalışan kuralların mantığını kavramak, SOC (Security Operations Center) veya Malware Analiz mülakatlarında size doğrudan puan kazandıracak asıl teknik derinliktir.

Örnek 1: Anti-VM (Sanal Makine Kaçışı) Tespit Kuralı

Zararlı yazılımlar genellikle bir analiz ortamında (Sandbox) olduklarını anladıklarında kendilerini gizlemek için çalışmayı durdururlar. Kural dosyamızın adı: **anti-VM-Strings-targeting-VMWare.yml**.

Rule ↑↓	Address	Namespace ↑	ATT&CK Technique ↑↓	Malware Behavior Catalog ↑↓
reference anti-VM strings targeting VMWare		anti-analysis/anti-vm/vm-detection	Virtualization/Sandbox Evasion (T1497) ↳ System Checks (T1497.001)	Anti-Behavioral Analysis::Virtual Machine Detection [B0009]
▼ - or: - regex: /VMWare/i - "}VMWare"	file+0x487292			

CAPA Web Explorer'da bu yeteneğin tespit edildiğini gördüğümüzde, arka plandaki YAML kuralının yapısını incelemeliyiz.

```
rule:
  meta:
    name: reference anti-VM strings targeting VMWare
    namespace: anti-analysis/anti-vm/vm-detection
    authors:
      - michael.hunhoff@mandiant.com
      - "@johnk3r"
    scopes:
      static: file
      dynamic: file
    att&ck:
      - Defense Evasion::Virtualization/Sandbox Evasion::System Checks [T1497.001]
    mbc:
```

```

- Anti-Behavioral Analysis::Virtual Machine Detection [B0009]
references:
- https://github.com/LordNoteworthy/al-khaser/blob/master/al-
khaser/AntiVM/VMWare.cpp
examples:
- al-khaser_x86.exe_
-
b83480162ede09d4aa6d4850f9faa0a4c3834152752fd04cfdb22d647aa1f825:0x17D80
features:
- or:
- string: /VMWare/i
- string: /VMTools/i
- string: /SOFTWARE\\VMware, Inc\\.\\VMware Tools/i
- string: /vmnet\\.sys/i
- string: /vmmouse\\.sys/i
... (diğer sys ve dll dosyaları)
- string: /vmci/i
description: VMWare VMCi Bus Driver

```

Kuralın Teknik Çözümlemesi:

- meta : Bu blok, kuralın kimliğini tanımlar. Burada kuralın doğrudan **MITRE ATT&CK T1497.001 (Sistem Kontrolleri ile Sanallaştırma Kaçışı)** tekniğine haritalandığını (mapping) görüyoruz. Bu, raporlama yaparken doğrudan bu ID'yi referans gösterebileceğimiz anlamına gelir.
- features : Tüm teknik tespit mantığının yattığı yer burasıdır.
- or koşulu: Belirtilen özelliklerden (features) herhangi **birinin** bulunmasının kuralı tetiklemek için yeterli olduğunu belirtir.
- string: /VMWare/i : Dosya içerisinde "VMWare" dizgesini (string) arar. Buradaki kritik nokta sonundaki /i (case-insensitive) parametresidir. Zararlı yazılım geliştiricisi "vmware", "VMWARE" veya "vMwArE" yazarak tespitten kaçmaya çalışsa bile, /i bayrağı sayesinde Regex (Düzenli İfade) büyük/küçük harf duyarız çalışarak bu kaçış (Evasion) girişimini boşa çıkarır.
- CAPA Web Explorer tam olarak bu namespace (isim uzayı) altında çalışarak hedef dosyanın (cryptbot.bin) içerisinde bu belirtilen stringlerden birini (Örn: VMWare) bulmuş ve kuralı tetiklemiştir.

Örnek 2: Zamanlanmış Görev (Scheduled Task) ile Kalıcılık (Persistence) Tespit Kuralı

schedule task via schtasks	persistence/scheduled-tasks	Scheduled Task/Job (T1053) ↳ Scheduled Task (T1053.005)
<pre> function @ 0x438E25 - and: - match: host-interaction/process/create - or: - and: - regex: /schtasks/i - "schtasks.exe" - regex: /\create /i - " /create /tn \"ServiceData4\" /tr \"%s\" /st 00:01 /du 9800:59 /sc once /ri 1 /f" </pre>	<pre> 0x439954 0x4399CA 0x439993 </pre>	

Zararlı yazılımlar sisteme ilk erişimi sağladıktan sonra, sistem yeniden başlatıldığında bile hayatta kalmak için Kalıcılık (Persistence) mekanizmaları kurarlar. Zamanlanmış görevler (Scheduled Tasks) Windows ortamında bunun en yaygın yoludur. İlgili kural dosyamız: **schedule task via schtasks.yml**.

```
rule:
  meta:
    name: schedule task via schtasks
    namespace: persistence/scheduled-tasks
    authors:
      - 0x534a@mailbox.org
    scopes:
      static: function
      dynamic: thread
    att&ck:
      - Persistence::Scheduled Task/Job::Scheduled Task [T1053.005]
    examples:
      - 79cde1aa711e321b4939805d27e160be:0x401440
  features:
    - and:
      - match: host-interaction/process/create
      - or:
        - and:
          - string: /schtasks/i
          - string: /\create /i
        - string: /Register-ScheduledTask /i
```

Kuralın Teknik Çözümlemesi ve Mantıksal Zinciri:

- Bu kural doğrudan MITRE **T1053.005 (Zamanlanmış Görev/İş)** tekniği ile eşleşmektedir.
- `features` bloğu bu kez çok daha karmaşık bir mantık kapısı (`and` , `or` kombinasyonu) içeriyor.
- En dışta bir `and` (VE) mantığı var. Bu şu anlama gelir: Sistemin bu kuralı "Zararlıdır" olarak işaretlemesi için iki ana koşulun **aynı anda** gerçekleşmesi gerekir.
- Koşul 1:** `match: host-interaction/process/create` -> Zararlı yazılımın işletim sistemi üzerinde yeni bir süreç (Process) başlatma yeteneğine sahip olması gerekir (Örn: `CreateProcessAPI` çağırısı).
- Koşul 2:** Bir `or` (VEYA) bloğu içeriyor. Zararlı yazılım ya `Register-ScheduledTask` (PowerShell Cmdlet) string'ini içermelidir **VEYA** bir alt `and` bloğuna girerek hem `/schtasks/i` (Windows komut satırı aracı) hem de `/\create /i` (Yeni görev oluşturma parametresi) stringlerini aynı anda içermelidir. `\` kullanımı, regex formatında bölü `/` işaretinden kaçış (Escape) yapmak içindir.
- Sonuç olarak CAPA aracı, `cryptbot.bin` dosyasının belleğinde `schtasks` ve `/create` argümanlarını tespit etmiş, aynı zamanda bir süreç oluşturma (`process create`) potansiyeli taşıdığını onaylamış ve bu zincirleme tespit sonucunda "Bu zararlı,

zamanlanmış görev üzerinden kalıcılık sağlıyor" hükmüne varmıştır. Web Explorer üzerinde ilgili özelliğe tıklandığında gösterilen sonuç tam olarak bu koşulların doğrulandığının kanıtıdır.